Звіт HW22. Тестування продуктивності REST-додатку у JMeter

1. Що тестували і як

Додаток: QA Pro REST App (`oleksandrgolubishko/qa_pro_rest_app`), працює в Docker на `http://localhost:3001`. **Інструмент:** Apache JMeter 5.5, запуск у non-GUI режимі. **Комп'ютер:** AMD Ryzen 5 4600H (6 ядер / 12 потоків), 16 ГБ ОЗП, Windows. JMeter і сервер працюють на одному комп'ютері.

Що робить кожен «користувач» у тесті (одне коло):

1. `GET /characters` - отримати список усіх персонажів
2. `POST /character` - створити персонажа
3. `GET /character/{id}` - отримати цього персонажа
4. `PUT /character/{id}` - змінити його
5. `DELETE /character/{id}` - видалити його

Потім коло повторюється, доки не мине час тесту.

2. Етапи тест-плану і чому саме так

Етап	**Що зроблено**	**Чому**
Адреса сервера	Host, Port, Protocol винесені в змінні + HTTP Request Defaults	щоб змінити адресу в одному місці
Параметризація	CSV Data Set Config читає файл `characters.csv` (200 рядків: ім'я, вік, нове ім'я, новий вік)	щоб кожен запит мав різні дані, як у реальному житті
Кореляція	JSON Extractor бере `id` зі відповіді на POST і кладе у змінну `charId`; Regex Extractor бере `name`	сервер сам придумує `id`, тому потрібно взяти його з відповіді й використати в наступних запитах (GET, PUT, DELETE)
Захист від каскаду помилок	If Controller: GET/PUT/DELETE за `id` виконуються, лише якщо POST дав `id`	у ДЗ 21 після невдалого POST `id` був 0, і виникали тисячі зайвих помилок 404
Перевірки (Assertions)	Код відповіді (201 для POST, 200 для інших), наявність `id` у JSON,	щоб рахувати помилкою не лише збій, а й неправильну

Етап	Що зроблено	Чому
	Duration Assertion: відповідь не довша за 1 с	або надто повільну відповідь
Пауза між запитами	Constant Timer (Think time)	щоб імітувати людину, яка не клацає без перерви
Listeners	Aggregate Report, Summary Report, Graph Results; View Results Tree вимкнено	дерево результатів сильно навантажує JMeter і спотворює тест
Графіки	HTML Dashboard (`-e -o`): час відповіді, пропускна здатність, потоки тощо	вони вбудовані в JMeter, плагіни не потрібні
Чистий стан	`docker restart` перед кожним прогоном	у ДЗ 21 список персонажів ріс із кожним тестом і спотворював результати
Моніторинг ресурсів	`typeperf` (CPU і пам'ять комп'ютера) і `docker stats` (CPU і пам'ять контейнера)	замість PerfMon: працює без додаткових плагінів

3. Три сценарії навантаження

Сценарій	Потоків (користувачів)	Розгін (ramp-up)	Тривалість	Пауза
baseline (звичайне)	10	5 с	60 с	500 мс
load (підвищене)	100	20 с	120 с	200 мс
stress (екстремальне)	1000	60 с	180 с	100 мс

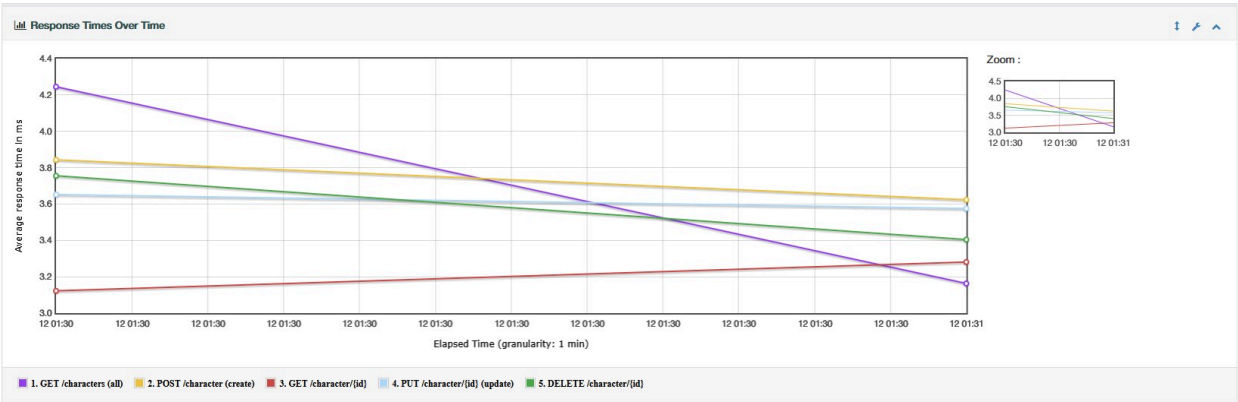
4. Результати

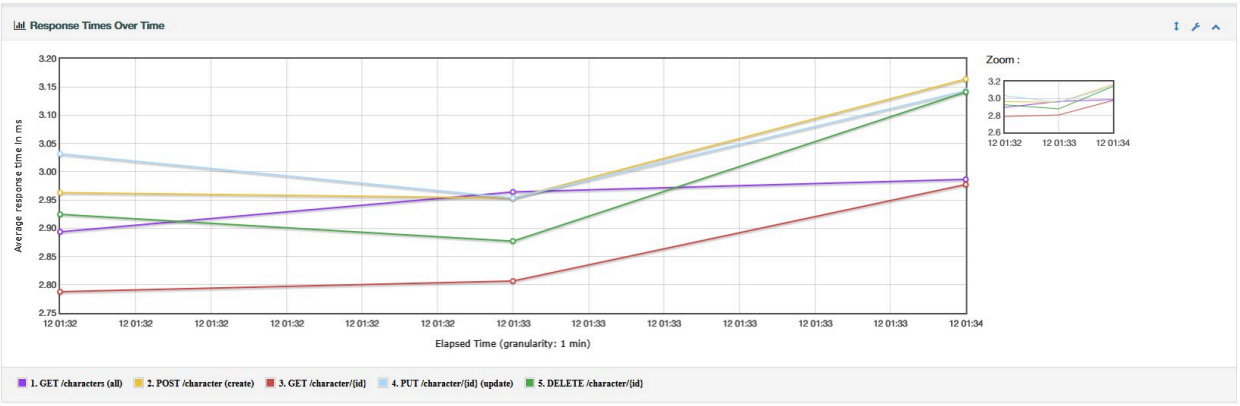
Сценарій	Запитів	Помилок	Середній час, мс	95% запитів швидше, мс	99% запитів швидше, мс	Найдовший, мс	Запитів за секунду
baseline	1 135	0 (0%)	3,48	5	7	35	19,2
load	53 978	0 (0%)	2,94	7	12	42	451

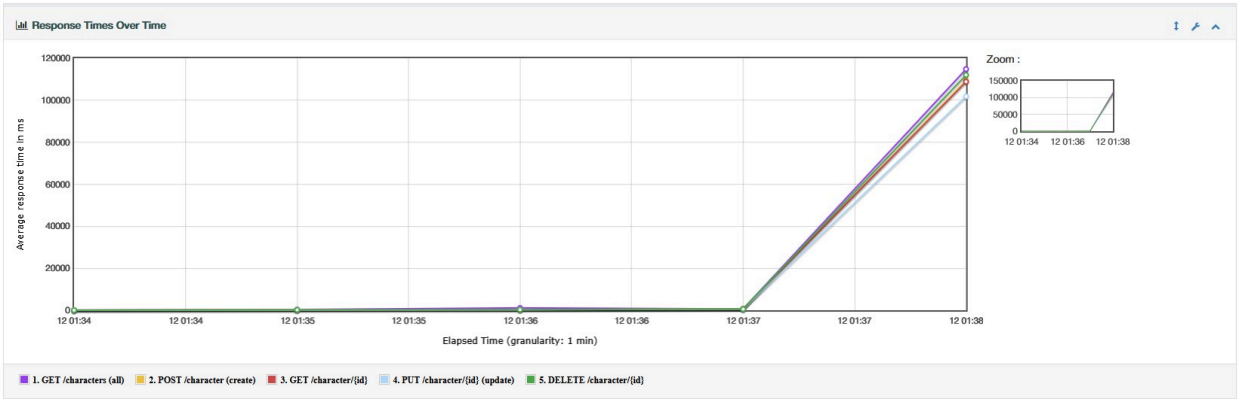
Сценарій	Запитів	Помилок	Середній час, мс	95% запитів швидше, мс	99% запитів швидше, мс	Найдовший, мс	Запитів за секунду
stress	282 814	2 531 (0,89%)	446	172	57 096	150 642	1 160

Графіки

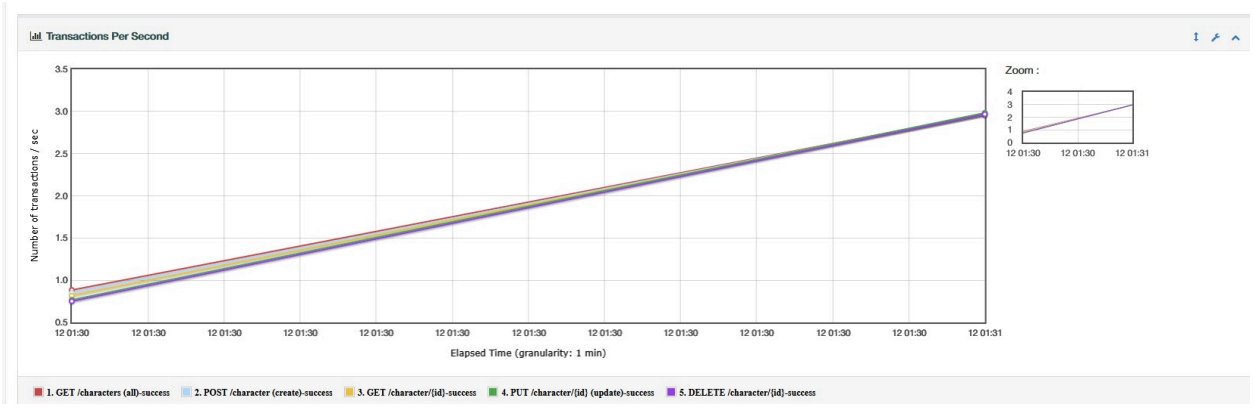
- Response Times Over Time (час відповіді в часі): baseline, load, stress



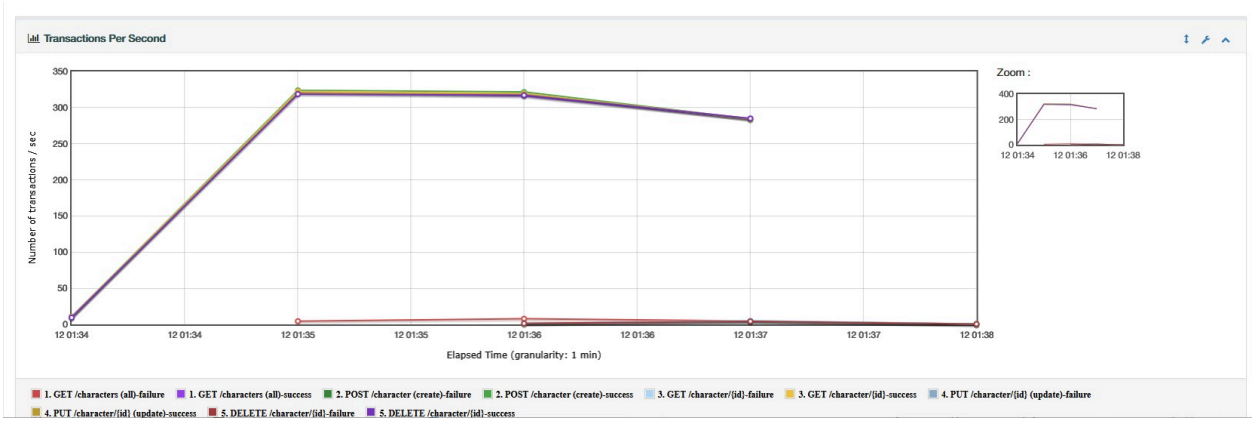




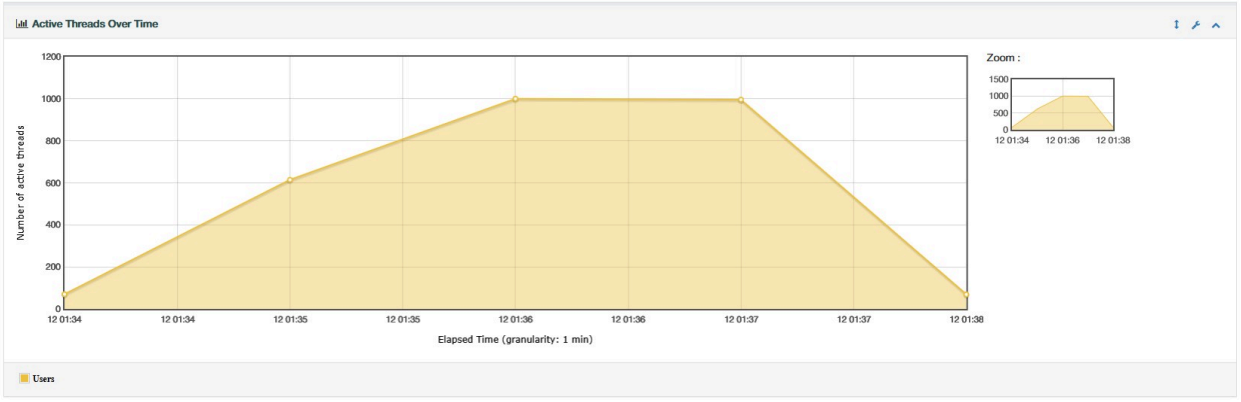
- Transactions Per Second (запитів за секунду): baseline, load, stress



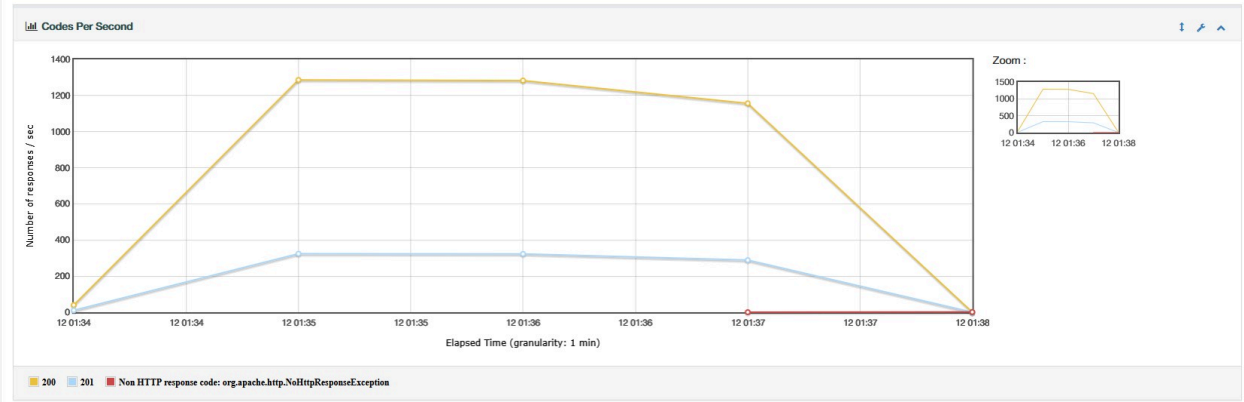




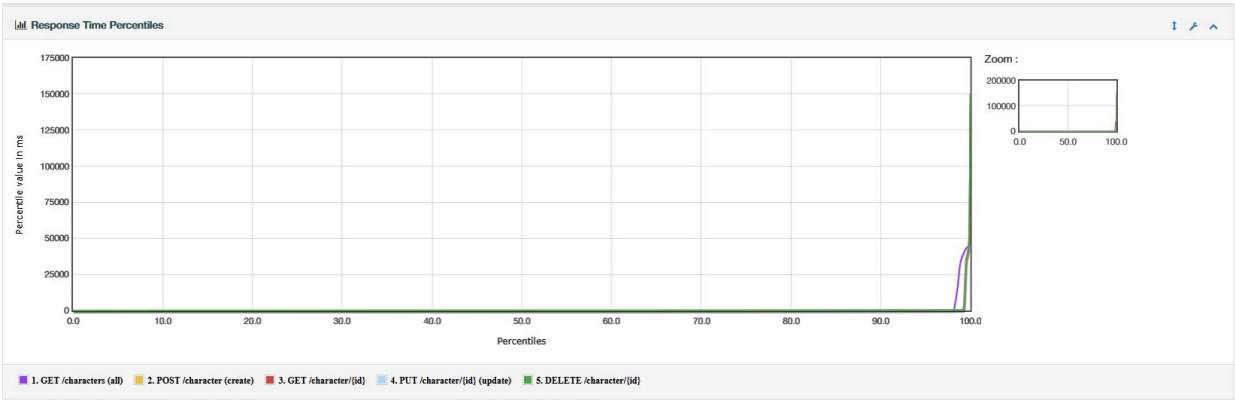
- Active Threads Over Time (кількість потоків): stress



- Codes Per Second (коди відповідей): stress



- Response Time Percentiles: stress



5. Ресурси під час stress

Комп'ютер загалом (`stress_resources.csv`, перші 3 хвилини тесту):

- процесор у середньому 43%, максимум 65%, жодного разу не було ≥ 90%;
- пам'ять зайнята приблизно на 65-67%, майже рівно.

Контейнер із сервером (`stress_docker.csv`):

- процесор контейнера майже весь тест тримався на **102-108%**. Це означає, що **одне ядро повністю зайняте**;
- пам'ять контейнера близько 97 МБ із доступних 7,4 ГБ, тобто дуже мало.

6. Аналіз простими словами

1. **Звичайне й підвищене навантаження сервер тримає легко.** На 10 і на 100 користувачів помилок немає, а відповіді приходять за кілька мілісекунд (95% швидше за 7 мс). У тесті на 100 користувачів кількість запитів за секунду (451) обмежена не сервером, а паузами між запитами.
2. **На 1000 користувачів з'являється проблема, і вона не в середньому часі, а у «довгому хвості».** Половина запитів швидше за 68 мс, 95% швидше за 172 мс, але найповільніший 1% відповідав десятки секунд (99% швидше за 57 с, максимум 150 с). Тому середнє значення (446 мс) нічого не каже, дивитися треба на перцентилі.
3. **Звідки взялися помилки (2 531 із 282 814, тобто 0,89%):**

    - **Сервер не відповів зовсім** (`NoHttpResponseException: failed to respond`). У POST усі 364 помилки саме такі. Сервер закрив з'єднання без відповіді.
    - **Відповідь правильна (код 200), але надто повільна.** В `GET /character/{id}` частина помилок має код 200 і нормальне тіло, але відповідь прийшла приблизно через 44 секунди (перевірка на ліміт 1 с спрацювала). Ці запити почалися майже одночасно в багатьох потоках і чекали разом: сервер на якийсь час «завис», а потім відповів усім.
4. **Найважчий запит: `GET /characters`.** Він повертає весь список, тому відповідь росте разом із кількістю персонажів. У stress він був найповільнішим (середній час 716 мс проти ~370 мс в інших) і отримав найбільше даних (4 019 КБ/с проти 25-66 КБ/с в інших запитів).
5. **У чому вузьке місце.** Процесор комп'ютера був вільний (до 65%), пам'ять теж. Зате **процесор контейнера майже весь час був на 100% одного ядра**. Сервер обробляє запити в одному потоці, тому не може використати інші ядра й упирається в одне ядро. Це схоже на причину стелі приблизно **1 200 запитів за секунду** і на зависання. Це найімовірніше пояснення за метриками; без коду сервера стверджувати на 100% не можна.

7. Рекомендації

Для сервісу:

- запускати кілька екземплярів/процесів сервера й розподіляти між ними запити (балансувальник), щоб використати більше ніж одне ядро;
- зробити `GET /characters` по сторінках (пагінація) або обмежити розмір відповіді;
- обмежити кількість одночасних з'єднань і час очікування, щоб запити не висіли по 40-150 секунд.

Для самих тестів:

- запускати JMeter на окремому комп'ютері, щоб він не конкурував із сервером за ресурси;
- не вмикати View Results Tree під час навантажувальних прогонів;
- перед кожним прогоном повертати дані в чистий стан (перезапуск контейнера);
- у графіках ставити дрібнішу деталізацію (1 секунда), щоб бачити зависання.